

04 — Operations

This document covers running and administering a MegaMOO server: starting it, configuring it, the permission model, persistence and backups, and the JSON API / MCP integration that lets external tools (including AI assistants) drive the live world.

- [Requirements](#)
- [Starting the server](#)
- [Command-line reference](#)
- [Creating a database](#)
- [Configuration](#)
- [The permission ladder](#)
- [Player administration](#)
- [Persistence and backups](#)
- [Logging](#)
- [Shutdown and restart](#)
- [The JSON API](#)
- [The MCP integration](#)

Requirements

- **Python 3.10+.** No `pip install` is needed for the core server — it runs on the standard library alone.
- **Optional:** `bcrypt` for stronger password hashing (a salted SHA-256 fallback is built in); `websockets` for browser clients; `mcp` for the AI integration bridge.

Starting the server

The entry point is `megamoo.py` :

```
python3 megamoo.py sf.db
```

That loads the database `sf.db` and listens for Telnet on the **default port 6770** (`DEFAULT_PORT` in `moo/globals.py`). Connect with any MUD client or:

```
telnet localhost 6770
```

You'll see the splash screen and the prompt `Enter your username or NEW to create a new account: .`

Port note: the project README's quick-start uses `7777` as an *example* value, but the actual built-in default is `6770`. Pass `--port` (or a positional port) to choose explicitly.

The argument parser supports both a LambdaMOO-style positional form and modern flags, and is smart about positional arguments — a second positional that is all digits is treated as a port, a hostname-like token as the bind host:

```
python3 megamoo.py sf.db                # db, default host/port
python3 megamoo.py sf.db 7777            # db on port 7777
python3 megamoo.py sf.db localhost 7777 # db, bind localhost, port 7777
python3 megamoo.py --input sf.db --port 8888 --host 127.0.0.1
```

On startup the server prints a banner with the version, loads the database, opens the listener(s), restores tickers, and starts its background tasks (checkpointing, the task queue, the ticker loop).

Status: the engine is at beta `0.7.0` and has **not yet been load- or play-tested**. Treat it as pre-release: suitable for development and single-developer worlds, not yet for an open multiplayer deployment. The version shown is `SERVER_VERSION` in `moo/globals.py`, the single source of truth for the build.

First login: the owner/wizard account (#100)

The shipped database includes **#100**, the default owner/wizard account — the first character in the game. It has full authority (`gm5`, plus the `WIZARD` and `PROGRAMMER` flags), so it can create players, build rooms, write verbs, and administer the server. Log in with the default credentials:

- **Username:** `Wizard`
- **Password:** `wizard`

Two things to do on first login, before anything else:

1. **Change the password.** The default is public knowledge. Use `password` (interactive) or `setpass <newpassword>` from the OOC lobby. **Do this immediately** in any environment reachable by others — it is the first item on the [security checklist](#).
2. **Make it yours — rename #100.** This account is meant to *become* the owner's own character, not stay a generic "Wizard." Rename it with `@name #100 = <YourName>` (see naming objects); the login username follows the account name. From here you grant other staff their tiers with `@auth` (see [the permission ladder](#)).

Keep at least one account at gm5 . If you rename and re-secure #100 it remains your god account; only create additional gm5 accounts deliberately.

Command-line reference

Flag	Default	Purpose
database (positional) / --input , -i	— (required)	Database file to load.
new_database (positional) / -- output , -o	—	Create a new database from the template (see below).
port (positional) / --port , -p	6770	TCP port for the Telnet listener.
--host	0.0.0.0	Bind address.
--config , -c	—	Path to a ServerConfig JSON file (see Configuration).
--api	off	Enable the JSON API server.
--api-port	7778	API server port.
--api-token	—	Shared secret API clients must present.
--log-level	INFO	DEBUG , INFO , WARNING , or ERROR .
--version , -v	—	Print the version and exit.
-h , --help	—	Full help.

Creating a database

Give the server a **template** plus an **output path** and it will build a fresh database from the template and exit, telling you how to launch it:

```
python3 megamoo.py template.db newworld.db
# ...creates newworld.db, then:
python3 megamoo.py newworld.db
```

Pointing the server at a database file that doesn't exist (with no output path) is an error — it won't silently create an empty world; you start from a template.

The shipped database. MegaMOO ships with a default database containing the core object hierarchy — every object through **#54**, plus **#90** (BaseEdible) and **#91** (BaseDrinkable) — and **#100**, the owner/wizard account and the first character in the game. That gives you the base prototypes (rooms, exits, containers, furniture, wearables, consumables) and a usable login from the first launch; build your world on top of it.

Configuration

Two distinct files are easy to confuse:

- **config.json in the repo root is *not* the server config.** It configures an unrelated document/vector-search tool (embeddings model, chunk size). The server does not read it.
- **The server's `--config` expects a `ServerConfig` JSON** with `network`, `database`, `protocol`, and `api` sections, matching the dataclasses in `moo/config.py`. Anything omitted falls back to the documented default.

A representative config (all values shown are the defaults):

```

{
  "network": {
    "host": "0.0.0.0",
    "port": 6770,
    "max_connections": 100,
    "connection_timeout": 3600,
    "max_players": 0,
    "ssl_enabled": false,
    "websocket_enabled": false,
    "websocket_port": 8888
  },
  "database": {
    "path": "game.db",
    "checkpoint_interval": 3600,
    "max_checkpoints": 10,
    "transaction_log_enabled": true,
    "auto_save_interval": 300,
    "compress_checkpoints": true,
    "backup_on_start": true,
    "max_object_cache": 1000
  },
  "protocol": {
    "...": "MXP / GMCP / MSDP / MSSP toggles"
  },
  "api": {
    "enabled": false,
    "port": 7778,
    "host": "127.0.0.1",
    "auth_token": "",
    "testbot_objnum": 0
  }
}

```

Notes on the fields that matter most in practice:

- **network.max_players: 0** means unlimited; **max_connections** bounds raw sockets.
- **database.auto_save_interval (5 min)** is the in-memory-to-disk save; **checkpoint_interval (1 hr)** writes pruned recovery snapshots; the two are independent. **backup_on_start** takes a full copy before the server begins writing.
- **api.host defaults to 127.0.0.1** — keep it that way unless you really mean to expose the API, and always set a strong **auth_token**. **testbot_objnum** pins the API's driver character (see [MCP](#)).

The config validates on load: SSL requires a cert/key, an enabled API requires a valid port, etc.

The permission ladder

Authorization is a five-level ladder stored in each character's `auth` property and managed in-game. Two object flags are kept in sync automatically: granting `gm3` sets `PROGRAMMER`, granting `gm4` sets `WIZARD`.

Level	Name	Can, roughly
gm1	AssistantGM	Inspect (<code>+show</code>), teleport, basic helpers.
gm2	Builder	Full world-building: rooms, exits, objects, descriptions, messages.
gm3	Coder	Programming: create from any parent, add/remove properties and verbs, <code>reparent</code> , <code>eval</code> , inspect internals.
gm4	Admin	Player administration (<code>@delplayer</code>).
gm5	God	Server control (<code>@shutdown</code> / <code>@restart</code>), renumbering, number reservation, granting auth.

Grant and inspect with `@auth` (gm5 only):

```
@auth bob = add gm2
@auth bob = remove gm3
@auth bob = list
```

Per-verb authorization is independent of who *owns* a verb: `@verbauth` and the `auth=N` option on `@adverb` set the minimum level required to invoke a specific verb (see Building Worlds).

Player administration

Command	Level	What it does
<code>@auth <player> = add\ remove <level> / = list</code>	gm5	Manage a player's auth levels (syncs <code>PROGRAMMER</code> / <code>WIZARD</code>).
<code>@delplayer <object#></code>	gm4	Permanently delete an account: moves its IC characters and inventories to storage (#8), clears local properties, returns the account to the player pool (#2). Requires typing <code>YES</code> to confirm.

Persistence and backups

State lives in **SQLite** with WAL journaling. In normal operation you do not issue saves — property writes are persisted through automatically (see Architecture). For operations:

- **Auto-save** flushes memory to disk every `auto_save_interval` seconds (default 300).
- **Checkpoints** are written every `checkpoint_interval` seconds (default 3600) into a `db_checkpoints/` directory, with `max_checkpoints` retained and optional gzip.
- **backup_on_start** copies the database before the server begins writing.
- The WAL files (`*.db-wal`, `*.db-shm`) alongside the database are normal SQLite artifacts; don't delete them out from under a running server.

To take a manual backup, the safest path is to `@shutdown` (which saves and unpuppets everyone) and copy the `.db` file. A live file copy can be torn if the server is mid-write — acceptable for a scratch snapshot, not for an authoritative backup.

Logging

The server logs to `megamoo.log` in the working directory via a rotating file handler (10 MB per file, 5 backups) and also to the console. The level defaults to `INFO`; raise it with `--log-level DEBUG` when diagnosing. Because the log is a plain file, it can be tailed even when the server is down — the MCP bridge exposes exactly that (`tail_log`).

Shutdown and restart

Both are gm5 verbs and both save the database, broadcast an optional message, and unpuppet every connected character before disconnecting:

```
@shutdown Server going down for maintenance.  
@restart Applying a hotfix, back in a moment.
```

`@restart` re-executes the server process after a clean save, so it picks up engine code changes that require a full reload. You rarely need it for game content: verbs written in-game with `@program` are already live (hot-coded, no restart), and verbs edited on disk are pulled in with `@reload` — both without downtime. See Hot coding.

The JSON API

When started with `--api`, the server runs a newline-delimited JSON-over-TCP API (`moo/api.py`), **off by default and localhost-only** by default. It is the control plane for external tooling — editors, scripts, and the MCP bridge. Enable it with a token:

```
python3 megamoo.py sf.db --api --api-token <token>
```

The API authenticates with the shared `auth_token` and drives a dedicated character (`testbot_objnum` , auto-provisioned if unset — see below). Keep `api.host` at `127.0.0.1` unless you have a specific reason to expose it, and always set a strong token.

The MCP integration

MegaMOO ships an [MCP](#) bridge (`tools/megamoo_mcp.py`) so an MCP client — such as Claude Code — can drive the running game directly: run commands as a bot character, inspect and edit live objects and verbs, search, and read logs, without touching the database by hand.

```
Claude Code —stdio—> tools/megamoo_mcp.py —TCP/JSON—> ApiServer (port 7778)
                        (MCP bridge)                      inside running MegaMOO
                        |
                        └—reads—> megamoo.log (works even when server is down)
```

The bridge is a standalone FastMCP process; the game server gains no new dependencies.

One-time setup

```
pip install mcp
python3 megamoo.py sf.db --api --api-token <token>
claude mcp add megamoo -e MEGAMOO_API_TOKEN=<token> -- python3 ~/sfdev/tools/
    megamoo_mcp.py
```

Tools

Game tools (require the server running):

Tool	Description
<code>run_command</code>	Execute a game command as the bot character; returns its output (color stripped).
<code>disconnect_testbot</code>	Cleanly unpuppet the bot via the normal <code>on_unpuppet</code> path.
<code>get_object</code>	Full object info: name, parent, flags, properties, verbs.
<code>get_location</code> / <code>list_contents</code>	An object's location / a room or container's contents.
<code>list_verbs</code> / <code>get_verb</code> / <code>set_verb</code> / <code>delete_verb</code>	Read and write verb source live.
<code>list_properties</code> / <code>get_property</code> / <code>set_property</code>	Read and write properties.
<code>eval_code</code>	Evaluate arbitrary Python in the verb context.
<code>search_verbs</code> / <code>search_objects</code>	Full-text search across verb source / objects by name or description.

Disk tools (work even when the server is down):

Tool	Description
<code>tail_log</code>	Last N lines of <code>megamoo.log</code> , with an optional regex filter.
<code>server_status</code>	API reachability; if up, uptime and player count.

Notes

- The bot character is **auto-created on first `run_command`** if no character of the configured name exists under `#5 ICharacter`. To pin a specific character, set `api.testbot_objnum` before starting the server.
- The integration test suite (`python3 -m pytest tests/`) boots its own server on separate ports against a scratch copy of the database, so it can run alongside a dev server. Two caveats: the test subprocess appends to the shared `megamoo.log`, and the database snapshot is a plain file copy (could tear under heavy live writes). When in doubt, stop the server or run only the unit tests: `python3 -m pytest tests/ --ignore=tests/test_integration_api.py`.

Security checklist

Before exposing a server beyond your own machine:

1. **Change the default wizard password.** Log in as `Wizard` / `wizard` and change it immediately (see [First login](#)).

2. **Keep the JSON API local.** Leave `api.host` at `127.0.0.1` and set a strong `api.auth_token` ; only widen it deliberately.
 3. **Enable TLS** (`ssl_enabled` with cert/key) if the server is reachable over the internet.
 4. **Turn off debug mode** in production so internal state isn't exposed.
 5. **Right-size** `max_connections` for your hardware, and review the per-IP connection rate limit.
 6. **Keep** `backup_on_start` **on** (default) and set up **external backups** of the database.
 7. **Grant the minimum auth tier.** Give staff the lowest `gm` level that lets them do their job (see [the permission ladder](#)), and keep the set of `gm5` accounts small.
-

Next: Engine Systems — hooks, the ticker, and the effects system; then The Prototype Library.